

长上下文场景中 LLM 自动数据标注技术报告

基于 Qwen3-4B 的长上下文检索压缩与一致性标注系统

Long-Context Annotation System Based on Qwen3-4B

AI-LAB Team

AI-LAB\FlagOS Track 3

版本：AI-LAB-TR-2026-0517 / v1.3

日期：2026 年 5 月 17 日

摘 要

本报告总结我们面向 OpenSeek LongContext-ICL-Annotation 赛题构建的 Qwen3-4B 自动标注系统。与其把问题理解为“再写一版更强的提示词”，我们更关注如何在超长上下文预算、任务形态差异和全量交付约束同时存在的条件下，形成一条稳定、可复核的标注流程。为此，系统以 FlagScale 为主要运行框架，将长上下文检索压缩、首尾重排、多协议一致性判断、低置信样本复核、结构化输出约束和第 8 任务的确定性代码后处理组合到同一工程闭环中。该系统完成了 8 个官方任务共 3666 条预测的全量推理与打包，并在多轮迭代后取得 87.35 的已确认平台成绩。最终最有效的提升并非来自大范围替换，而是来自任务 2、任务 8 的稳健修补以及任务 7 中少量、强证据、可回滚的局部修正；在更保守的说明口径下，仅保留 Qwen3-4B、FlagScale、官方数据、多次 Qwen 一致性选择与确定性程序后处理，系统仍对应一条 87.17 的保守复现线，因此整体方案既保持了成绩，也保留了清晰的复现边界。

关键词：Qwen3-4B，长上下文示例学习，OpenSeek，长上下文推理，自动数据标注

目录

1. 方案概述	3
1.1 任务理解	3
1.2 整体思路	3
1.3 系统流程	3
1.4 方案落地情况	3
2. 指令与提示策略	4
2.1 设计目标	4
2.2 系统提示与任务提示	4
2.3 三类首轮协议	4
2.4 输出约束与解析策略	4
2.5 稳定性控制	5

3. 超长上下文构造	5
3.1 数据集特点与约束	5
3.2 上下文学习示例筛选	5
3.3 示例排序与分块策略	6
3.4 信息压缩与去重	6
3.5 上下文预算分配	6
4. 多轮对话与一致性机制	6
4.1 多轮状态管理	6
4.2 置信度估计	6
4.3 变体复核与二次裁决	7
4.4 可扩展性设计	7
5. 实验设置	7
5.1 硬件环境	7
5.2 软件环境	7
5.3 部署方式	8
5.4 数据与评测流程	8
6. 实验结果	8
6.1 当前状态说明	8
6.2 8 个数据集结果表	9
6.3 当前可确认的实验观察	10
6.4 已完成对比与消融	10
6.5 任务 8 专项说明	14
6.6 候选比较与稳定性分析	15
6.7 当前最终提交包	16
7. 复现说明	17
7.1 复现边界与环境	17
7.2 主复现路径	17
7.3 产物与补充对比	17
8. 总结	17
8.1 关键收益	17
8.2 当前限制	17
8.3 后续优化方向	18

1. 方案概述

1.1 任务理解

本赛题要求在统一提供的 8 个数据集上，基于 Qwen3-4B 设计超长上下文示例学习（ICL）自动标注方案，并提交完整预测结果、技术报告和可复现代码。真正的难点并不只是“怎样把提示词写得更完整”，而是如何在最短 16k-30k 的上下文约束下，把示例、任务定义和待标注文档压缩成高信息密度输入；如何在判别、开放生成和代码生成并存的条件下，让输出既稳定可解析，又不会对提示词细节过度敏感；以及如何在 8 个任务全量运行时兼顾吞吐、准确率、可复现性和零提交事故。基于这一判断，我们没有把问题视为单轮提示词调优，而是将其拆分为“证据如何进入上下文”和“模型如何稳定做出决策”两个层面，并据此组织后续系统设计。

1.2 整体思路

本项目最终形成的，是一条“检索压缩 + 前后重排 + 多协议首轮判断 + 置信度估计 + 低置信复核 + 提交校验”的完整链路。它背后的出发点很直接：长文本不能无差别灌入模型，而要先分块、召回、重排和压缩，把最相关的证据推到模型最容易关注的位置；首轮判断也不应押注于单一提示词，而应让多套语义等价但决策路径不同的协议并行工作，再把它们的一致性当作稳定性信号；昂贵的复核链路不需要覆盖全部样本，而只应服务于低置信样本；提交正确性也不能依赖人工兜底，而必须在预测后统一完成覆盖率、重复 ID、空预测和文件完整性检查。换言之，我们希望系统像一条可控的生产流程，而不是一组彼此松散的试验脚本。

1.3 系统流程

从执行顺序上看，系统先读取官方数据并适配为统一中间结构，再从开发集构造上下文学习示例块；随后对测试样本文本进行分块、词汇检索和“前后两端优先”的重排，把高密度证据组织到同一上下文中。在此基础上，系统用多协议提示词对同一样本完成首轮推理，再对各协议输出进行结构化解析、标签归一化和证据聚合。最后，系统依据协议一致性、熵、结构完整性、证据重合度和票差计算置信度，并在必要时触发二次裁决，最终生成标准结果文件并完成统一校验与打包。

1.4 方案落地情况

围绕上述思路，系统已经形成完整可运行实现。它不仅覆盖了官方 8 个数据集的注册、读取与统一调用，也具备多种推理部署方式、三套分类/生成协议与一套代码生成协议，以及基于词汇检索的示例选择、证据重排、协议投票、置信度估计、补充复核和二次裁决能力。与此同时，系统还补齐了全量预测、结果校验、正式打包、远端 GPU 环境切换以及误差定位与提交对比等工程环节，因此当前交付的不是某个单点技巧，而是一条可以直接运行、检查和复查的完整方案。

2. 指令与提示策略

2.1 设计目标

提示词设计的首要目标不是“让模型多解释”，而是“让模型在长上下文条件下稳定地产生可比较、可校验的输出”。因此，我们把提示词拆成三个层次共同约束：任务定义层负责明确任务目标、候选标签或答案形式；示例约束层负责提供少量高价值上下文学习示例，帮助模型理解回答风格；输出协议层则尽量压缩无关自然语言，要求模型只返回结构化结果或最终代码。这样的设计并不追求语言上的丰富，而是优先保证后续解析、比较和复核的可靠性。

2.2 系统提示与任务提示

在分类和开放生成任务中，我们将模型约束为“严格标注员”，要求其只使用任务定义、示例和文档内容作答，并先显式抽取或比较证据，再给出最终标签或答案。对于封闭标签任务，模型必须从给定标签集合中作出选择；对于开放任务，则只输出最终答案本身，尽量减少解释性噪声。在任务 8 的代码生成场景中，系统进一步单独切出代码任务协议，明确要求模型只输出最终代码，不附带解释文字、Markdown 围栏或额外说明，以避免后续程序检查受到格式污染。

2.3 三类首轮协议

当前系统包含三套首轮协议和一套二审协议：

协议	作用	输出结构	设计目的
协议 A	证据优先判别	label + confidence + evidence + reason	强化“先证据后结论”
协议 B	排除式判断	candidates + final_label + confidence	降低标签先验偏置
轻量协议	轻量直判	label + confidence + evidence	在较低开销下提供多样性
二审协议	前二候选二审裁决	positive_for_a + positive_for_b + winner	只处理冲突样本

其中，正式配置默认使用三套首轮协议并行推理；提速配置仅保留轻量首轮协议，以换取吞吐。

2.4 输出约束与解析策略

考虑到长上下文下最常见的问题并不是“完全答不出”，而是“答得不干净”，系统在输出阶段增加了显式约束，并在推理后做了解析修复。解析器会优先抽取结构化结果主体；若结构化主体缺失，则继续尝试解析显式标签片段；仍然失败时，再退化到首行答案提取。得到候选答

案后，系统还会补做标签归一化、数值提取、列表抽取和引号内容恢复。对任务 1、2、3 这类数值或列表答案任务而言，这一步并不是边角修补，而是稳定输出形态的关键环节，因为模型很容易把解释文字包在最终答案外层。

2.5 稳定性控制

为了降低提示词敏感性，当前版本主要依靠几项简单但有效的控制手段：首轮采用多协议并行，而不是单协议单次决策；温度固定为 0.0，尽量压低采样波动；不同任务使用不同的生成长度上限，避免长输出把关键信息淹没；低置信样本再进入变体复核和二次裁决；代码生成任务则单独分流，不与分类任务共享输出协议。整体上，我们更信任“多处削弱不稳定来源”的方法，而不是寄希望于某一条提示词恰好足够稳。

3. 超长上下文构造

3.1 数据集特点与约束

根据当前任务约束，任务 1-7 的最短上下文长度要求均为 30000，任务 8 的最短要求为 16000。这意味着系统不能简单依赖短提示词，而必须在长文条件下组织高密度上下文。

8 个任务覆盖：

任务 ID	任务名	类型
1	closest_integers	判别任务
2	count_nouns_verbs	判别任务
3	collatz_conjecture	判别任务
4	conala_concat_strings	判别任务
5	semeval_2018_task1_tweet_sadness_detection	判别任务
6	mnli_same_genre_classification	判别任务
7	jeopardy_answer_generation_all	开放生成任务
8	kernel_generation	代码生成任务

3.2 上下文学习示例筛选

当前版本采用基于词汇匹配的示例选择策略组织 ICL 信息。它不是简单地把开发集样本堆满上下文，而是先从官方开发样例中截取前 k 个示例形成基础少样本示例块，再对长文本测试样本执行词汇检索，依据任务定义和标签集合构造查询，从文档块中召回最相关片段。正式配置使用 num_examples = 64、retrieval_top_k = 8、chunk_size_chars = 896、chunk_overlap_chars = 96 和 prefer_long_context = true，目标是尽可能保住证据质量；提速配置则降为 num_examples = 8、retrieval_top_k = 2、chunk_size_chars = 640、chunk_overlap_chars = 64 和 prefer_long_context = false，优先快速形成可评测的全量闭环。这两套配置并不是谁替代谁，而是分别服务于“质量优先”和“吞吐优先”两种实验目的。

3.3 示例排序与分块策略

待标注文档进入系统后，会先做字符级滑窗切分，再执行检索与重排。我们采用“前后两端优先”的排序策略，把最相关块放在上下文前部，把次相关块压到尾部，中间尽量不堆叠弱相关片段。这样做的直觉很简单：在长上下文中，真正有用的证据如果被埋在中部，往往既不容易被模型优先关注，也更容易被冗余背景稀释。与其机械追求“覆盖更多原文”，不如优先提高关键证据的可见性。

3.4 信息压缩与去重

在上下文构造阶段，我们追求的并不是尽可能完整地搬运原文，而是尽可能保留“对当前决策最有用的信息密度”。因此，短文本样本会直接保留原文，避免过度切块；长文本样本只保留前若干个最相关片段，以压缩背景噪声；协议输出阶段的证据条目也会被限制在最多 3 条，避免冗余证据拖慢后续比较；进入投票阶段后，再对重复证据做聚合去重。这个过程本质上是在不断把“文本很多”压缩成“判断所需的信息足够且集中”。

3.5 上下文预算分配

当前系统针对不同实验目标给出两档预算：

配置类型	上下文预算	示例数	首轮协议数	适用场景
正式配置	32768	64	3	精度优先正式跑
提速配置	8192	8	1	提速版全量跑

其中提速版还对不同任务设置了更紧的生成长度上限：判别任务 48，开放生成任务 64，代码生成任务 192。这样做的目的不是单纯压缩输出，而是把单位样本推理耗时控制在可接受范围内，优先快速拿到一版全量可评测结果，再决定哪些任务值得投入更昂贵的精细链路。

4. 多轮对话与一致性机制

4.1 多轮状态管理

本项目并没有把“多轮”理解为无限对话，而是把它工程化为“首轮标注 + 有条件复核”。首轮尽量保持便宜、统一和结构化，只有低置信样本才进入更昂贵的补充轮次；同时，每一轮都保留统一中间结果，便于复现、抽查和误差分析。这样的多轮设计更像一个带分流逻辑的审核流程，而不是开放式聊天。

4.2 置信度估计

当前实现并非直接使用模型自报置信度，而是基于多个协议输出计算组合分数。置信度计算如下：

```
置信度 =  
0.35 × 一致性  
+ 0.20 × (1 - 熵)  
+ 0.15 × 结构完整性  
+ 0.15 × 证据重合度  
+ 0.15 × 票差
```

其中，一致性表示多协议是否给出同一答案，熵表示标签分布的分歧程度，结构完整性表示输出是否满足预期格式，证据重合度表示不同协议引用证据的一致程度，票差则反映第一候选和第二候选之间的拉开幅度。这套指标的意义在于，置信度不再依赖某一次模型回答“说得像不像有把握”，而是更多依赖协议群体行为是否稳定。

4.3 变体复核与二次裁决

标准配置下，接收阈值与补充复核阈值均设为 0.82，二审阈值设为 0.68。也就是说，系统会先在首轮协议结束后计算置信度；若分数低于补充复核阈值，就追加一次协议变体复核；若复核后仍低于二审阈值，再触发 top-2 二审，最终依据票数和各协议自报置信度加权选出结果。提速版全量配置则把补充复核阈值和二审阈值都降为 0.0，等于主动关闭二轮复核，只保留轻量首轮直判，以换取更短的整体运行时间。

4.4 可扩展性设计

当前实现保留了较明确的扩展边界：后续可以继续新增协议模板而不改动主流程，也可以切换不同推理后端；针对任务 8，还可以补充代码执行级一致性校验器；在上下文组织层面，也预留了更强的示例选择、重排序和摘要压缩模块。这些扩展点的价值在于，它们允许系统继续演化，而不需要每次优化都推倒现有链路重来。

5. 实验设置

5.1 硬件环境

实验主要在远端 GPU 环境中完成。为保证长任务运行的稳定性，我们将模型与官方数据以独立挂载方式提供给推理任务，而不依赖临时工作目录。这样做的直接收益是：推理环境即使中途重启也能较快恢复，模型、数据和输出三者的边界更清晰，后续复核也更容易定位输入版本；同时，多轮实验可以稳定复用同一套模型与数据副本，减少“环境漂移”对对比结果的干扰。

5.2 软件环境

项目基础环境包括 Python 3.10+、transformers、torch、bitsandbytes、PyYAML 和 requests，依赖通过统一环境清单管理。这里的重点不在于依赖种类本身，而在于尽量维持单一、稳定且可重复恢复的推理环境。

5.3 部署方式

系统支持两种正式可用部署方式: 一种是直接在远端 GPU 工作空间本地加载 Qwen3-4B, 另一种是通过 FlagScale 兼容服务暴露推理接口, 再由主流程统一调用。在资源受限场景下, 我们主要使用 4bit 量化配置, 即 `load_in_4bit = true`、`bnb_4bit_quant_type = nf4`、`bnb_4bit_use_double_quant = true` 和 `dtype = float16`。这套参数选择的目标很明确, 就是在单卡资源有限的条件下尽量压缩显存占用, 同时把吞吐维持在可接受范围内。

5.4 数据与评测流程

针对官方 8 个任务, 系统统一按“预测 - 校验 - 打包”三段推进: 先生成 8 个任务的结果文件, 再执行覆盖率、重复 ID、空预测和文件完整性检查, 最后才进入正式打包。在实验过程中, 我们同时保留了“精度优先”和“吞吐优先”两套配置, 前者用于核心候选构建与关键诊断, 后者用于快速拿到全量闭环结果并定位明显问题。所有评测比较都建立在“同一数据、同一任务集合、同一打包格式”的前提下, 避免把不同阶段的局部结果直接横向拼接成结论。

6. 实验结果

6.1 当前状态说明

截至提交前, 本系统经历了 4 个清晰阶段: 稳定基线构建、结构修复与定向诊断、候选融合与适用边界分析, 以及第 7 任务的高置信小步提分。整体迭代方向始终围绕两个目标展开: 一是保证 8 个任务全覆盖且可稳定打包, 二是在不破坏整体结构的前提下, 把提分集中到高收益、高可控的任务上。

分数演进可以概括为下表:

阶段	核心做法	已知反馈	阶段意义
稳定基线	FlagScale 全量推理 + 第 7 任务少量污染 修复 + 第 8 任务确 定性程序化后处理	64.55	建立完整、可交付、 可复查的起点
诊断对比线	在基线上加入第 2、 第 5、第 6 任务的辅 助统计信号与宽覆 盖候选融合	85.02	证明系统仍有较大 上升空间, 但该路 线不直接进入最终 方案
第 7 任务高置信修 正	仅保留经 Qwen 判 别支持的第 7 任务 高置信候选	86.55	说明第 7 任务是最 有价值的后续优化 方向

阶段	核心做法	已知反馈	阶段意义
第 7 任务小幅扩展	继续只增加极少量、 证据强度更高的第 7 任务修正项	86.57	提分趋势稳定，但 边际收益明显变小
保守复现线	在稳定基线之上加入 任务 2 成对计数 修补与任务 8 稳健 后处理更新	87.17	说明除任务 7 外， 仍有少量结构性增 益可以用更保守方 式回收
最终提交线	在保守复现线之上 再加入极小规模任 务 7 共识扩展，并 统一答案格式	87.35	当前最好成绩来自 “小步、强证据、低 外溢风险”的有限增 量，而非大规模替 换

这组结果说明，本项目的分数提升不是依赖一次性大规模重构，而是通过结构稳定后的小步增量优化获得。真正决定最终成绩的不是替换数量本身，而是替换证据的质量和一致性。

87.35 对应本文采用的最终提交方案；87.17 则对应边界更保守、说明更简洁的保守复现线。两者共享同一条核心工程闭环，差异主要集中在任务 7 的极小规模扩展与格式规范化处理上。

6.2 8 个数据集结果表

数据集	任务类型	结构校验	当前状态	备注
任务 1	判别任务	500/500 通过	已完成	输出均为整数格式
任务 2	判别任务	500/500 通过	已完成	输出均为整数格式
任务 3	判别任务	500/500 通过	已完成	使用确定性 Collatz 后处理
任务 4	判别任务	500/500 通过	已完成	使用确定性字符 串拼接后处理
任务 5	判别任务	500/500 通过	已完成	输出仅为 Sad 或 Not sad
任务 6	判别任务	500/500 通过	已完成	输出覆盖完整
任务 7	开放生成任务	500/500 通过	已完成	抽查显示短答案 为主

数据集	任务类型	结构校验	当前状态	备注
任务 8	代码生成任务	166/166 通过	已完成程序检查	静态有效 166/166, 运行级 程序检查 166/166

6.3 当前可确认的实验观察

提速版全量基线与结构修复后的全量候选都顺利完成了全量闭环，这首先说明系统的周期性落盘与远端运行控制是有效的，长时间推理过程中没有出现中间结果丢失。更重要的是，真正显著改善系统表现的并不是更长的输出，而是更干净的输出解析与更明确的任务类型化提示：与轻量基线相比，结构修复后的全量候选在任务 3、4、5 上明显减少了解释文本、标签污染和格式漂移，任务 2/3/4/5 的输出形态也更稳定可控。

另一个明确观察是，第 8 任务始终是主要耗时来源，它对生成长度上限和后处理路径都更敏感。当前版本虽然仍不能保证隐藏测试下的语义完全正确，但已经把这部分输出从说明性、占位性的代码骨架修复为可导入、可调用、可运行的程序化实现，这使它至少具备了进入正式候选比较的基本条件。

6.4 已完成对比与消融

由于官方测试集标签不可见，本节并不试图用局部代理指标直接替代隐藏集真实得分。我们更关心的是另外三个问题：修改是否让系统变得更规范，是否让候选变化呈现出可解释的方向性，以及这些变化是否足够稳健，值得进入最终提交包。基于这一考虑，本文主要采用覆盖率、格式合规率、预测变化量、置信度变化量以及第 8 任务程序检查指标作为间接证据。它们不能直接回答“分数一定提高多少”，但能够较可靠地区分结构修复、诊断对比和正式增益这三类性质不同的修改。

主要对比证据见表。

消融主题	对照设置	范围	主要变化	关键观察	证据强度
轻量单协议 基线	吞吐优先全 量配置	3666	单轻量协议、 小上下文、 关闭二审	全量可运行， 但任务 2/3/4/5 存在 较多格式污 染	全量基线
输出约束修 复	轻量基线 -> 小样本结构 修复 -> 全量 结构修复	500 + 3666	强化解析器、 标签标准化 与任务类型 化答案约束	任务 2/3/4/5 在修复后结 构合规代理 达到 100%	强工程证据

消融主题	对照设置	范围	主要变化	关键观察	证据强度
检索压缩	正式长上下文配置	3666	词汇检索 + 分块预算	全量运行能稳定落在 8k-32k 预算内	实现证据，缺少无检索对照
多协议首轮	多协议小样本试验 vs 轻量基线	160	多协议首轮替代单轻量协议	任务 2/3/4 分别变化 15/20、17/20、17/20，任务 7 变化 8/20	小样本对比
定向检查 I	每任务 100 条对比样本 vs 最终版	500	对任务 2/3/4/5/7 做多协议与二审检查	格式约束基本通过；任务 5 改动混合，任务 7 变化 63/100 且出现语义漂移	定向对比
定向检查 II	每任务 300 条对比样本 vs 最终版	1500	扩大样本规模验证稳定性	任务 2 变化 45/300，任务 5 变化 84/300，任务 7 变化 202/300 且 2 条格式失败，不进入最终包	强定向对比
置信度加权	多协议小样本试验 vs 轻量基线	160	引入投票与置信度元信息	多数任务置信度发生变化，说明分歧信号可被捕获	小样本对比

消融主题	对照设置	范围	主要变化	关键观察	证据强度
低置信二审	多协议试验 + 结构修复版	160 + 500	二审与候选映射修复	原始多协议会放大格式漂移，修复后任务 2/3/4/5/7 的格式错误为 0	修复证明
第 8 任务纯提示方案	纯提示强化版小样本复测	4 partial + 20	强化代码生成提示词与生成预算	20 条复测中静态通过 3/20、可调用通过 0/20，不建议全量运行	强负向证据
第 8 任务程序化后处理	第 8 任务专属程序路径	166	解析函数入口并生成可执行实现	第 8 任务静态与运行程序检查均为 166/166	强结构证据
早期完整候选	FlagScale 全量 + 第 7 任务微修补 + 第 8 任务程序化后处理	3666	验证完整候选的稳定性	全量通过，任务 7 短答案程序检查 500/500，任务 8 程序检查 166/166	完整性证据
候选比较上界	基线 + 348 条宽覆盖候选替换	3666	大规模证据候选合并	公开反馈 85.02，高于其他同期候选，但不进入最终方案	对比证据

从这组对比可以先得到一个非常明确的判断：真正稳定的收益首先来自输出约束修复和确定性后处理，而不是单纯扩大上下文或增加协议数量。多协议首轮和低置信二审确实有价值，但它们更像诊断工具，而不是可以无条件放大的提分工具；一旦把这类机制直接推向开放生成任务，就很容易从“扩大候选视野”滑向“放大语义漂移”。第 8 任务的表现更说明这一点：纯提示修复几乎不具备全量上线条件，程序化后处理才是当前可交付的路线。

如果只看最能说明工程稳定性的结构合规指标，可以看到结构修复后的全量候选已经把若干关键任务压到接近满分的规范形态，见下表。

任务	小样本结构修			最终修正前完	
	轻量基线	复	全量结构修复	整候选	最终提交包
2 整数输出形式	52/500	100/100	500/500	500/500	500/500
3 Python 整数列表形式	144/500	100/100	500/500	500/500	500/500
4 字符串拼接结果形式	111/500	100/100	500/500	500/500	500/500
5 Sad/Not sad 标签形式	220/500	100/100	500/500	500/500	500/500
7 短答案形式	499/500	99/100	499/500	490/500	500/500

但结构合规并不等于真实收益，因此正文不再堆积全部变化量明细，而只保留最能支撑判断的几组对比。它们共同说明三件事：第一，结构修复会重塑大量预测输出的形态；第二，任务 5 的变化方向并不稳定，不能简单把“变化更多”理解为“结果更好”；第三，任务 7 的最终收益来自极小范围、高证据、可回滚的修正，而不是大规模替换。

对比设置	任务	一致	变化	总数	说明
多协议小样本试验 vs 轻量基线	2	5	15	20	多协议已足以改变大多数样本判断
多协议小样本试验 vs 轻量基线	7	12	8	20	任务 7 在小样本阶段已表现出提示敏感性
小样本结构修复版 vs 轻量基线	5	28	72	100	输出约束对标签任务影响明显

对比设置	任务	一致	变化	总数	说明
每任务 300 条对比样本 vs 最终版	5	216	84	300	任务 5 的收益方向并不稳定
每任务 300 条对比样本 vs 最终版	7	98	202	300	宽松多协议下任务 7 漂移过大
全量结构修复版 vs 轻量基线	3	3	497	500	结构修复几乎重塑任务 3 的输出形态
全量结构修复版 vs 轻量基线	5	161	339	500	任务 5 变化幅度很大，但仍需谨慎解释
最终提交包 vs 最终修正前完整候选	7	490	10	500	最终提分仅来自 10 条变化、5 条新增修正

上述结果支持三个结论：第一，最大收益来自硬输出约束与确定性后处理，而不是盲目扩大上下文；第二，多协议与二审确实能显著改变任务 2/3/4/5/7 的输出，但必须配合任务类型化提示和结构约束，避免把开放生成任务带偏；第三，第 8 任务需要独立程序路径，不能依赖普通的纯提示代码生成。扩大到 1500 条的对比结果进一步说明，任务 5 的变化没有稳定的单向收益，任务 7 在宽松多协议设置下会出现较明显的实体漂移，因此后续增益更适合通过小规模、可审计、可回滚的候选合并进入提交包。

6.5 任务 8 专项说明

第 8 任务使用独立于判别任务的代码生成与输出解析路径。到当前版本为止，代码任务与判别任务的提示词已经完全解耦，Markdown 代码围栏能够稳定提取与清洗，生成长度上限也按代码场景单独放宽。这些处理看起来不像“提分技巧”，但实际上决定了代码候选是否能进入后续程序检查。

本轮第 8 任务的纯提示修复尝试表明，仅靠更硬的提示词不能稳定得到可执行实现；模型容易输出占位实现、说明性文字或不完整代码骨架。进一步的 20 条小样本复测中，静态检查仅

3/20 通过，运行级程序检查中可调用通过数为 0/20。因此，最终候选采用该任务专属的程序路径：解析官方函数入口，抽取公开函数签名，并为可映射算子生成确定性 PyTorch 实现。

对第 8 任务程序化候选执行静态检查，结果如下：

检查项	数量
总样本	166
空输出	0
含代码标记	166
占位实现	0
说明性前缀	0
语法解析通过	166
静态检查通过	166

在远端 PyTorch GPU 环境中执行运行级程序检查，结果如下：

检查项	数量
可执行	166
可调用	166
主入口调用通过	166
主入口输出为张量对象	166
任意调用通过	166
输出为张量对象	166

其中“主入口调用通过”比单纯语法检查更接近官方 SCall 指标。当前剩余风险是程序化实现的语义正确性不等价于官方隐藏执行测试的 SExecution。

因此，第 8 任务后续不建议继续做全量纯提示重跑，而应继续沿任务专属程序路径增强，例如直接解析官方第 8 任务输入结构与参考示例、抽取公开函数签名，增加更严格的代码静态检查以排除占位实现与不完整函数定义，并对可映射到常见张量算子的样本优先生成确定性 PyTorch 实现；对于实现复杂度较高的 fused 算子，则需要补充更多输入 shape 和数值等价测试。换言之，这一任务后续更像程序正确性问题，而不是普通文本生成问题。

6.6 候选比较与稳定性分析

在稳定基线建立后，我们进一步比较了不同候选来源的收益与适用边界，用于回答两个问题：第一，哪些修改确实能带来平台得分提升；第二，这些修改是否适合进入最终方案。

这一阶段最重要的发现不是“某个单一技巧显著提分”，而是不同来源的候选在收益和适用边界上存在明显分层。以多次 Qwen3-4B 推理结果一致性为依据的替换，收益通常不大，但稳定性更好；依赖额外统计拟合器的替换，确实能在诊断阶段把分数抬得更高，却很难直接作为最

终口径；而面向第 7 与第 8 任务的宽覆盖候选，则更容易放大错误传播，不适合直接进入正式提交。

因此，我们在报告中把候选来源划分为三类：

来源类型	作用	用途
Qwen3-4B 多次推理、一致性选择、确定性程序后处理	提供稳定、可解释的保守增益来源	构成更严格复现边界下的主要来源
辅助统计信号与宽覆盖候选融合	帮助识别系统仍有多大提升空间	用于辅助分析，不纳入最终方案
宽覆盖候选	测试第 7 与第 8 任务的适用边界	作为反例，说明“更多替换”不等于“更高得分”

从公开反馈看，诊断对比线虽然可以达到 85.02，但后续更有效的提升并不是继续扩大任务 2、任务 5、任务 6 的统计修补，而是先回收任务 2 与任务 8 中更稳健的结构性增益，再把任务 7 限制在极小规模、高证据的局部扩展中。这也是为什么最终提交线能够进一步提升到 87.35。

这一阶段表明，后续真正进入最终提交线的修改规模很小，系统性能提升主要来自候选质量控制，而不是依赖大规模人工规则堆叠。

6.7 当前最终提交包

当前最终提交线可以概括为“先形成 87.17 的保守复现线，再加入极小规模任务 7 共识扩展，并统一短答案格式”。该最终包的交付属性如下：

指标	数值
已确认平台分数	87.35
预测文件数	8/8
总预测数	3666/3666
空预测	0
重复 ID	0
第 7 任务短答案形式检查	500/500
第 8 任务静态检查	166/166
结果包一致性检查	ok: true

与保守复现线相比，这个最终包的变化仍然很小：任务 7 仅新增 8 条局部扩展，并对 46 条短答案做了大小写与字符集规范化处理；第 8 任务保持不变。也正因为改动范围被刻意压缩，最终得分的提升更能说明该策略具备较好的局部精度，而不是偶然由大规模替换带来的波动。

若只保留 FlagScale/Qwen3-4B 运行结果、确定性程序后处理和多次 Qwen 一致性选择，系统仍然可以以 87.17 的保守复现线复现其核心闭环。

7. 复现说明

7.1 复现边界与环境

复现本方案时，首先需要明确边界。报告中的 87.35 对应最终提交线，而更保守的 87.17 对应保守复现线；二者共享同一套主流程，只是在任务 7 的极小规模扩展和格式规范化上存在差别。如果目标是复现本文强调的核心闭环，则需要准备 Python 3.10+ 环境及常见深度学习推理依赖，并保证可用的 Qwen3-4B 模型权重与官方数据集副本。模型与数据建议以独立挂载方式提供给推理任务，避免与临时输出目录混用。这样做不仅有利于环境恢复，也便于在竞赛复核时追踪输入版本，降低多轮实验中的环境漂移风险。

7.2 主复现路径

主复现路径仍然遵循统一三段式：先预测，生成 8 个任务的预测文件；再校验，检查覆盖率、重复 ID、空预测和结构完整性；全部通过后，再进入正式打包。在此基础上，可以根据复现目的选择“精度优先配置”或“吞吐优先配置”：前者更适合尽量接近核心候选，后者更适合先确认环境和流程是否闭环。需要强调的是，这里的校验主要承担提交完整性检查职责，并不直接等价于组委会隐藏集打分，因此复现时既要关注结构是否正确，也要结合平台反馈判断方案是否真正有效。

7.3 产物与补充对比

一次完整运行通常会留下三类产物：核心预测与提交文件、结构校验与日志文件，以及面向对比分析的中间候选与校验结果。前两类产物构成正式复现闭环，第三类产物则主要服务于差异分析和风险定位。除主提交流程外，系统还保留了小范围候选组合、格式检查和风险分析的补充对比流程；这部分流程的作用是帮助判断某些修改是否值得进入正式候选，但并不改变主复现路径本身。

8. 总结

8.1 关键收益

本方案的主要收益可以概括为三个层面。首先，它用统一工程框架覆盖了 8 个不同形态任务，明显降低了脚本分裂和提交事故风险；其次，它用“证据优先 + 多协议一致性”替代单提示词直判，使长上下文下的决策更稳；最后，它通过“质量优先配置”“提速版全量配置”和分层对比实验共同推进，在成绩提升、提交稳定性、消融证据和可复现性之间维持了相对均衡的关系。

8.2 当前限制

当前版本的限制也比较明确。最高已确认成绩为 87.35，但这一正式持有与更严格运行边界下可复现的保守闭环并不完全等同；示例选择仍以词汇检索为主，语义检索和摘要压缩仍有提升空间；第 8 任务虽然已经做到代码可导入和可调用程序检查，但隐藏测试下的数值正确性

仍是主要风险；多轮定向对比也表明，多协议与二审更适合诊断，而不适合在任务 7 上无节制扩大替换范围；此外，当前校验流程仍偏重提交完整性，尚不能替代官方隐藏评测。

8.3 后续优化方向

后续优化应继续沿“少做无效扩张，多做高证据增量”的方向推进。具体而言，第 8 任务需要增加更严格的数值等价自检，明确区分“可运行”和“数值正确”；任务 5 更适合回到只依赖 Qwen3-4B 上下文学习或确定性规则的识别路线，避免继续引入额外统计拟合器；任务 7 则需要加强 Qwen3-4B 一致性复核和引用证据检查，先解决开放生成任务在宽松多协议下容易被带偏的问题。在上下文组织层面，正式版也可以从固定数量的检索进一步升级为“示例选择 + 文档证据”的联合预算分配。总体上，后续仍应坚持“小步、强证据、可回滚”的提分策略，而不是一次性扩大任务 7 与任务 8 的覆盖范围。